Разворачивание Spring Boot приложений (nginx, systemd, firewall, journalctl)

Виктор Лоскутов
Java Team Lead, Сбер



Проверка связи

Если у вас нет звука:

- убедитесь, что на вашем устройстве и на колонках включён звук
- обновите страницу вебинара (или закройте страницу и заново присоединитесь к вебинару)
- откройте вебинар в другом браузере
- перезагрузите компьютер (ноутбук) и заново попытайтесь зайти

Поставьте в чат:

\+ если меня видно и слышно

\— если нет

Рекомендации

→ Если смотрите с компьютера

- Используйте браузеры Google Chrome или Microsoft Edge
- Если есть проблемы с изображением или звуком, обновите страницу — **F5**

→ Если смотрите с мобильного телефона или планшета

- Перейдите с мобильного интернет-соединения на **Wi-Fi**
- Если есть проблемы с изображением или звуком, перезапустите приложение на телефоне
- Предварительно проверьте, подходит ли ваше устройство для подключения к вебинару, по ссылке

Виктор Лоскутов

О спикере:

- Java Developer
- Java Team Lead, Сбер



Цели занятия

- Понять, как **Spring Boot**-приложение разворачивается и работает в **Linux**
- Разобраться в ролях **Docker**, **systemd** и **Nginx**
- Научиться управлять сервисом через **systemd**
- Научиться использовать **journalctl** для анализа логов и ошибок запуска
- Понимать типовые проблемы при запуске и эксплуатации сервиса

План занятия

1. Что значит «развернуть» **Spring Boot**-приложение
2. **Docker**: запуск и изоляция приложения
3. **Systemd**: управление жизненным циклом сервиса
4. **Nginx**: доступ к приложению извне
5. Итоги

Что значит «развернуть» Spring Boot-приложение



Что происходит после того, как приложение написано?

Развернуть приложение — значит

- запустить его в рабочей среде
- обеспечить стабильную и предсказуемую работу
- сделать его доступным для пользователей
- уметь управлять его состоянием и ошибками

Где живёт приложение после разработки

1

Запускается на сервере
работает в Linux-среде

2

Существует как процесс

3

Использует порты, файлы,
сеть

! Сервер — это не IDE и не ноутбук разработчика

Что требуется от рабочего сервиса

1. Автоматически запускаться при старте сервера
2. Корректно перезапускаться при сбоях
3. Быть доступным извне
4. Оставлять логи для диагностики

! «Запустилось один раз» ≠ «работает как сервис»

Какие задачи нужно решить при развертывании

1. Запустить приложение изолированно
2. Управлять его жизненным циклом
3. Организовать доступ пользователей
4. Контролировать сетевые ограничения
5. Понимать причины ошибок
6. Собирать и просматривать логи

Docker: запуск и изоляция приложения



Docker — первый инструмент

Зависимость приложения от окружения

При запуске приложения важно учитывать, что:

- Версия **Java** может отличаться
- Библиотеки и конфигурация отличаются
- Операционная система может быть другой
- Окружение разработчика и сервера не совпадает



Источник

Выход есть — использовать контейнер

Контейнеризация приложения

Контейнеризация — это подход к запуску приложений, при котором:

- Приложение запускается в изолированной среде
- Окружение запуска фиксируется и не зависит от сервера
- Приложение работает как отдельный процесс
- Одинаково запускается в разных средах

Идея контейнеризации — запускать приложение вместе с его окружением



Источник

Контейнеризация реализуется разными инструментами (**Docker**, **Podman**)

В рамках занятия используем **Docker** как стандарт де-факто

Что предлагает Docker

- **Единое окружение запуска.** Приложение всегда запускается с одинаковой версией **Java**, библиотеками и настройками
- **Изоляцию от операционной системы хоста.** Приложение не зависит от того, что установлено на сервере
- **Воспроизводимость запуска.** Одинаковый запуск на ноутбуке разработчика, тестовом и production-сервере
- **Переносимость приложения.** Приложение можно переносить между серверами без переустановки окружения
- **Контроль версии окружения.** Изменения окружения фиксируются и управляются так же, как код

Источник



Основные термины Docker

- **Docker Engine** — сервис, который запускает и управляет контейнерами
- **Docker Image (образ)** — шаблон приложения с окружением (**Java**, библиотеки, настройки)
- **Docker Container (контейнер)** — запущенный экземпляр образа, в котором работает приложение
- **Dockerfile** — инструкция, по которой собирается образ
- **Docker Registry** — хранилище образов (например, **Docker Hub**)

Как приложение попадает в Docker-образ

1

Docker читает Dockerfile

Построчно выполняет инструкции (**FROM**, **COPY**, **RUN**, **CMD**)

2

Формируется слоистый образ (layers)

Каждый шаг создаёт отдельный слой

Слои кешируются и переиспользуются

3

Приложение и окружение попадают в image

Java, библиотеки, конфигурация

Image становится неизменяемым артефактом

4

Image готов к запуску на любой машине

Без повторной настройки окружения

Пример Dockerfile для Spring Boot

```
FROM eclipse-temurin:21-jre

WORKDIR /usr/local/app

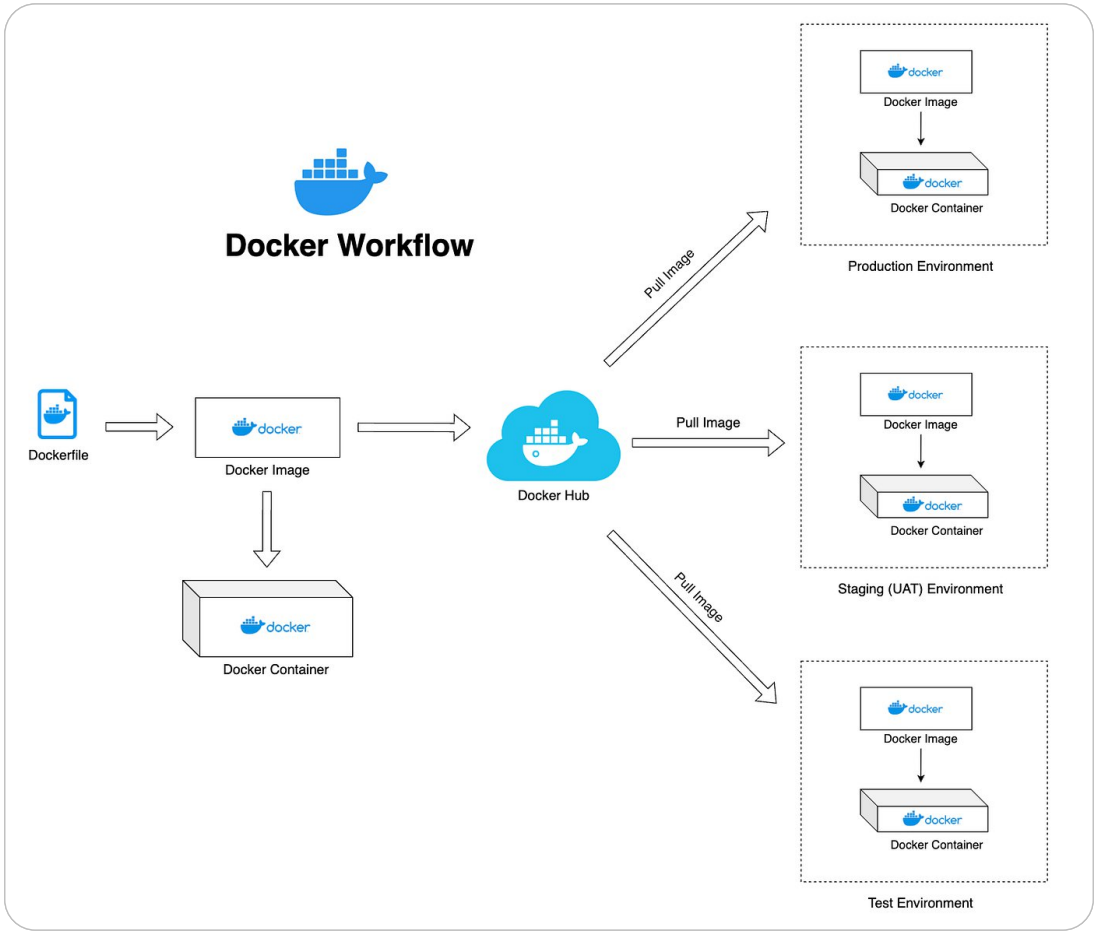
COPY target/*.jar app.jar

EXPOSE 8080

RUN useradd -m app
USER app

CMD [
  "java",
  "-Xms256m",
  "-Xmx512m",
  "-Dspring.profiles.active=prod",
  "-jar",
  "app.jar"
]
```

Docker Workflow



Как работает Docker

1

Dockerfile
Описать требования к запуску приложения: базовый образ, зависимости, команды запуска

2

Build → Image
Собрать Docker image — упаковку приложения вместе с окружением

3

Tag & Push → Registry
Сохранить image в registry для хранения и распространения

4

Pull на сервер
Загрузить image из registry на нужную машину

5

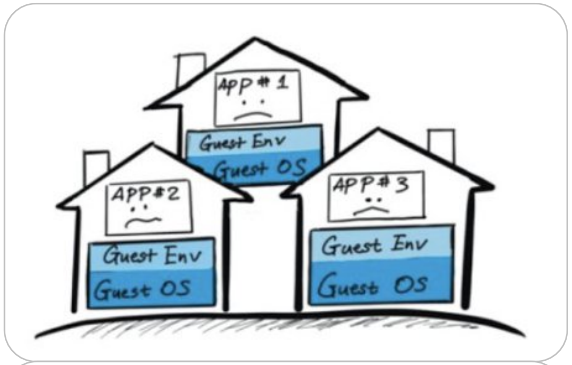
Run → Container
Запустить container из image и стартовать процесс приложения

6

Жизненный цикл и логи
Управлять контейнером, просматривать логи и обновлять версию приложения

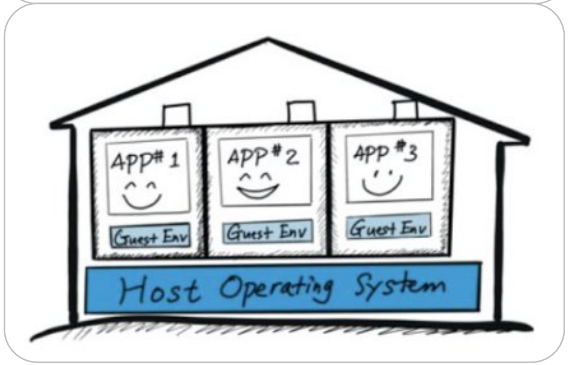
Docker vs Virtual Machine

Контейнеры и виртуальные машины часто путают, потому что обе технологии используются для запуска приложений в изолированной среде. Но на самом деле они решают задачу на разных уровнях.



Virtual Machine — изолирует целую операционную систему:

- Запускать гостевую ОС
- Использовать гипервизор
- Иметь собственное ядро ОС
- Запускать приложение внутри ОС



Docker изолирует приложение:

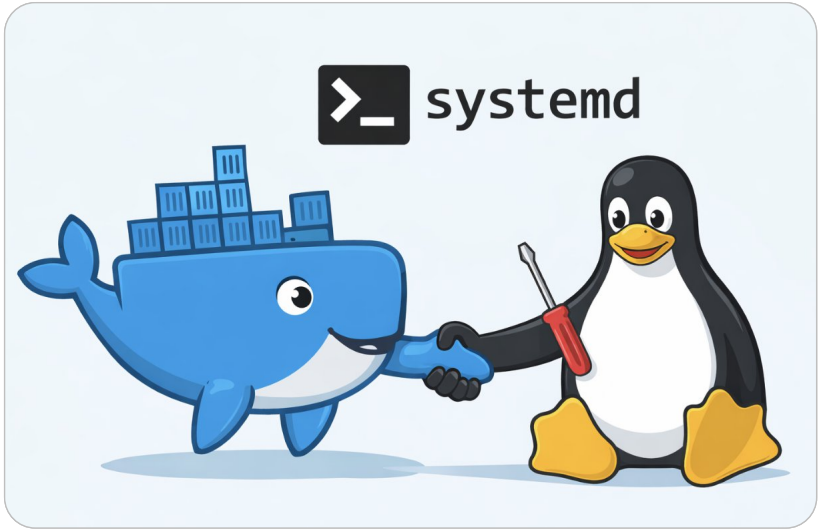
- Использовать ядро хостовой ОС
- Не запускать отдельную ОС
- Запускать приложение как процесс
- Упаковывать зависимости вместе с приложением

Что Docker не решает

Docker отлично решает задачу упаковки и запуска приложения, но не предназначен для управления его жизненным циклом на сервере

Docker не отвечает за:

- Управление состоянием сервиса (запущен / упал)
- Запуск приложения при старте сервера
- Определение порядка запуска сервисов



Источник

Systemd: управление жизненным циклом сервиса



systemd — инструмент управления сервисами

Что такое система инициализации

Система инициализации — это компонент операционной системы семейства Unix / Unix-like, который:

- Запускает системные и пользовательские сервисы
- Управляет жизненным циклом процессов (старт, остановка, перезапуск)
- Отслеживает состояние сервисов
- Обрабатывает зависимости между сервисами
- Обеспечивает готовность системы к работе после загрузки

Система инициализации

После завершения загрузки ядра в **Linux** и других системах семейства Unix начинается этап инициализации системы

Первый процесс в системе — **PID 1**. В Linux почти все процессы создаются путём клонирования родительского процесса. Однако существует одно важное исключение

init — это общее название для контроллера инициализации операционной системы, а не конкретная программа

Инициализация

- Запускаются системные сервисы и демоны
- Подготавливается пользовательское окружение
- Система приводится в рабочее состояние

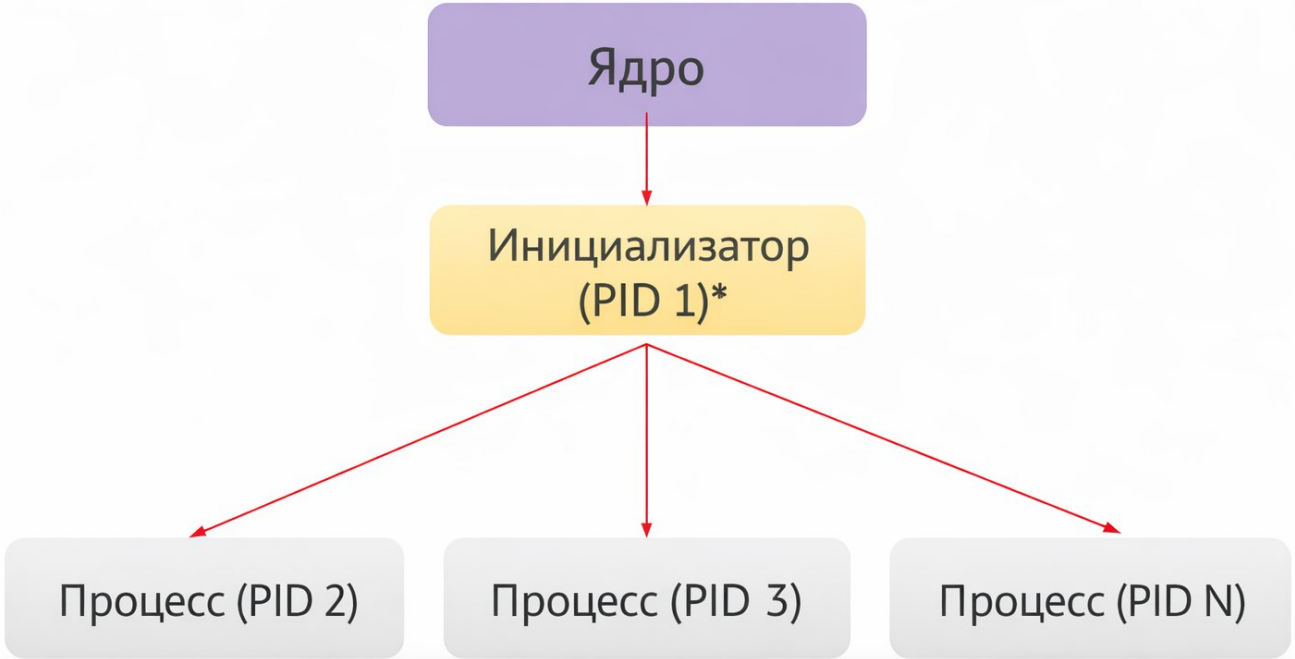
PID 1

- Создаётся самим ядром
- Процессу всегда назначается **PID 1**
- Отвечает за дальнейший запуск всех остальных процессов
- Является **init**

init

- Запускается первым в пользовательском пространстве
- Управляет запуском сервисов
- Контролирует жизненный цикл процессов
- Является «родителем» всех процессов в системе

Системы инициализации



Все пользовательские процессы в системе запускаются под управлением PID 1.

Виды систем инициализации

Система инициализации — это не одна конкретная технология. В разные периоды времени в Unix / Linux использовались разные реализации

Основные виды систем инициализации:

- **SysVinit (SysV).** Классическая система инициализации, основанная на shell-скриптах и последовательном запуске сервисов
- **systemd**. Современная система инициализации, использующая декларативные конфигурации и управление зависимостями. Наиболее распространённая система инициализации в современных Linux-дистрибутивах

Ограничения SysVinit

SysVinit — классическая система инициализации, которая долгое время использовалась в Unix/Linux. Однако с ростом сложности систем у неё появились серьёзные ограничения

Основные проблемы SysVinit:

- **Линейный запуск сервисов**. Сервисы запускаются последовательно, без учёта реальных зависимостей
- **Отсутствие явной модели зависимостей.** Порядок запуска определяется именами скриптов, а не логикой системы
- **Слабый контроль состояния сервисов.** Система не знает, запущен сервис или завершился с ошибкой
- **Нет автоматического перезапуска.** Упавший процесс не перезапускается без внешнего вмешательства
- **Проблемы с надёжностью запуска.** Возможен двойной запуск сервиса или ситуация, когда сервис не стартует вовсе

systemd — современная система инициализации

systemd — это современная система инициализации и управления сервисами в **Linux**

- Запускается первым в пользовательском пространстве (**PID 1**)
- Управляет запуском системных и пользовательских сервисов
- Управляет запуском системных и пользовательских сервисов
- Следит за состоянием процессов
- Автоматически перезапускает сервисы при сбоях
- Обеспечивает параллельный запуск компонентов системы

systemd не просто запускает сервисы, он управляет ими на протяжении всей жизни системы. Можно сказать, что это менеджер сервисов, а не просто механизм старта

Основные сущности systemd

systemd оперирует объектами, называемыми юнитами (**units**)

Основные типы unit-файлов:

- **service** — сервис (демон, приложение)
- **target** — логическая группа сервисов (аналог **runlevel**)
- **timer** — периодический запуск задач
- **socket** — запуск по сетевому событию
- **mount** — точки монтирования





Каждый юнит описывается конфигурационным файлом

Источник

Unit-файл systemd

Unit-файл описывает:

- Что запускать
- Когда запускать
- От чего зависит
- Как реагировать на сбои

Представляет собой текстовый файл следующего формата:

[Название секции в квадратных скобках]
имя_переменной=значение

Пример логических блоков:

- `[Unit]` — описание и зависимости
- `[Service]` — как запускать процесс
- `[Install]` — как включать в автозапуск

```ini
[Unit]
Description=My Java Application
After=network.target

[Service]
Type=simple
User=app
WorkingDirectory=/opt/myapp
ExecStart=/usr/bin/java -jar app.jar
Restart=always
RestartSec=5

[Install]
WantedBy=multi-user.target
```

systemctl — инструмент управления systemd

systemctl — это основной CLI-инструмент для управления **systemd,** чтобы

- Запускать и останавливать сервисы
- Перезапускать сервисы
- Включать и отключать автозапуск
- Проверять состояние сервисов
- Просматривать логи и ошибки

systemctl — интерфейс между администратором и **systemd**

Базовые команды systemctl

- **`systemctl start service`** — запустить сервис
- **`systemctl stop service`** — остановить сервис
- **`systemctl restart service`** — перезапустить сервис
- **`systemctl status service`** — проверить состояние
- **`systemctl enable service`** — включить автозапуск
- **`systemctl disable service`** — отключить автозапуск

Все сервисы управляются одинаковым образом, независимо от их типа

Логи в systemd (journalctl)

journalctl — это инструмент для просмотра логов **systemd**

- Собирает логи сервисов
- Хранит их централизованно
- Позволяет быстро найти причину ошибки
- Если сервис не запускается — **journalctl** в помощь для анализа

Логи в systemd (journalctl)

Основные команды **journalctl**:

- **`journalctl -u myapp.service`** — показать логи конкретного сервиса
- **`journalctl -u myapp.service –f`** — следить за логами в реальном времени (аналог tail -f)
- **`journalctl -xe`** — показать последние ошибки системы
- **`journalctl --since today`** — показать последние ошибки системы

Nginx: доступ к приложению извне



Приложение запущено
Что дальше?

Доступ пользователей к приложению

Приложение успешно запущено и работает как процесс

Однако само по себе оно:

- Слушает только локальный порт (например, 8080)
- Не умеет работать с доменами и HTTPS
- Не предназначено для прямого доступа пользователей
- Не обрабатывает большое количество входящих соединений
- Не управляет безопасностью и маршрутизацией запросов

Нужен компонент, который будет принимать внешние HTTP-запросы
и передавать их приложению

Таким компонентом является **Nginx**

Зачем нужен Nginx

Nginx — это высокопроизводительный веб-сервер, reverse proxy и балансировщик нагрузки, который стоит между пользователем и приложением

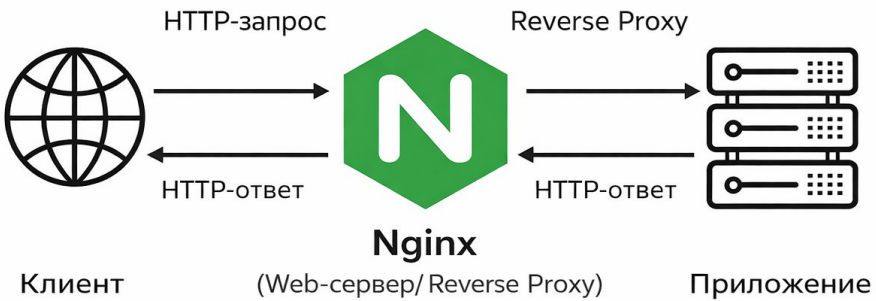
Он принимает все внешние запросы и управляет тем, как и куда они попадают

Nginx берёт на себя задачи:

- Принимать **HTTP / HTTPS**-запросы от пользователей
- Работать с доменами и портами
- Терминировать **SSL (HTTPS)**
- Проксировать запросы к backend-приложению
- Ограничивать и фильтровать трафик
- Обрабатывать большое количество соединений
- Отдавать статические файлы



Где находится Nginx в архитектуре приложения



- Пользователь обращается к серверу по **HTTP / HTTPS**
- **Nginx** принимает внешний трафик
- **Nginx** обрабатывает домены, **SSL** и маршрутизацию
- **Nginx** проксирует запросы во внутреннее приложение
- Приложение не доступно напрямую из интернета
- **Nginx — входная точка в систему**

Как Nginx обрабатывает запрос

Пользователь отправляет HTTP / HTTPS-запрос
Браузер обращается к серверу по доменному имени например: https://netology.ru

1

Запрос попадает в Nginx
Nginx принимает соединение, определяет домен (server_name), выбирает нужную конфигурацию

2

Nginx обрабатывает протокол и безопасность
Nginx обрабатывает HTTPS (SSL/TLS), проверяет настройки безопасности, может применять ограничения и фильтры

3

Nginx решает, куда направить запрос
Nginx отдаёт статический файл (HTML, CSS, изображения) или проксирует запрос во внутреннее приложение (например, на localhost:8080)

4

Приложение формирует ответ
Backend-приложение обрабатывает бизнес-логику, формирует HTTP-ответ, возвращает его обратно в Nginx

5

Nginx возвращает ответ пользователю
Nginx: принимает ответ от приложения, при необходимости модифицирует заголовки, отправляет итоговый ответ клиенту

6



Nginx управляет входящими запросами и защищает приложение от прямого доступа

Как настраивается Nginx

Nginx настраивается с помощью конфигурационных файлов, которые описывают:

- На каких портах принимать запросы
- Куда и как направлять трафик

Где (обычно) находятся конфиги:

- **/etc/nginx/nginx.conf** — основной файл
- **/etc/nginx/conf.d/*.conf** — дополнительные конфиги
- **/etc/nginx/sites-enabled/** — включённые сайты (Debian / Ubuntu)

Основные уровни конфигурации:

- **http** — глобальные HTTP-настройки server
- **server** — виртуальный сервер (домен / порт), отвечает на вопрос «для какого сайта?»
- **location** — правила обработки конкретных URL, отвечает на вопрос «что делать с этим URL?»

Конфигурация Nginx

1. **http — глобальный уровень — задаётся поведение Nginx как HTTP-сервера**
   - Общие HTTP-настройки
   - Применяются ко всем сайтам

2. **server — виртуальный сервер**
   - Определяет сайт (домен + порт)
   - Nginx выбирает server по server_name и listen

3. **location — маршрутизация запросов**
   - Определяет, что делать с конкретным URL
   - Может: проксировать в приложение, отдавать статику, возвращать ошибку, делать редирект

Пример конфигурации

```
http {
    # http — глобальные HTTP-настройки
    include       mime.types;
    default_type  application/octet-stream;

    server {
        # server — виртуальный сервер (домен + порт)
        listen 80;
        server_name example.com;

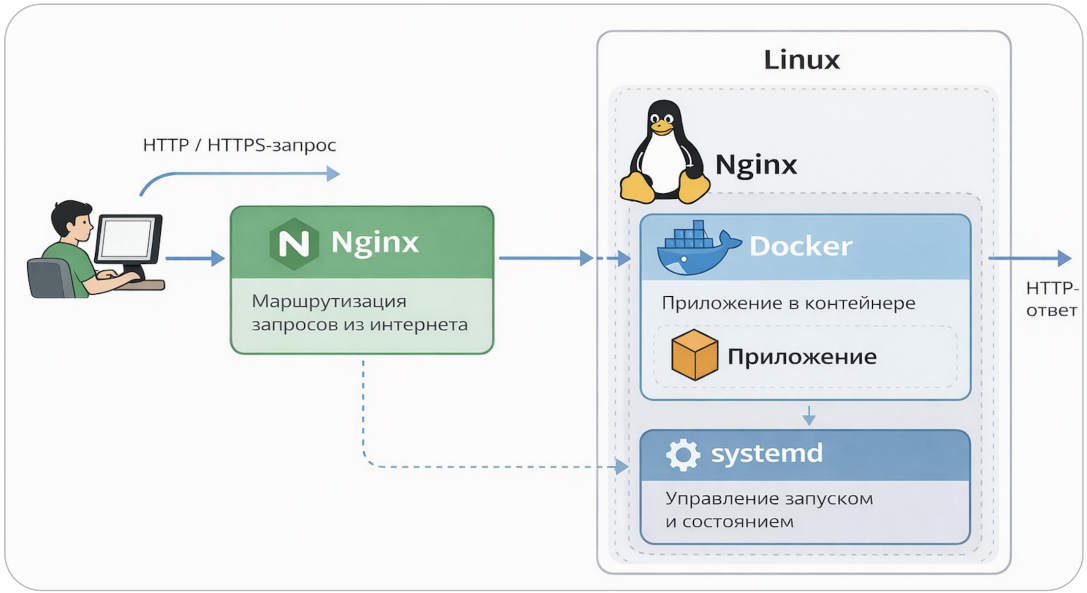
        location / {
            # location — правила обработки URL
            proxy_pass http://localhost:8080;
        }

        location /static/ {
            root /var/www/html;
        }
    }
}
```

Как все компоненты работают вместе

Как это работает на самом деле

- **Docker** отвечает за «что запускать» и «в каком окружении»
- **systemd** отвечает за «когда» и «в каком состоянии» работает приложение
- **Nginx** отвечает за «как пользователи попадают в приложение»



Ваши вопросы

Итоги

Итоги

- → Поняли, как **Spring Boot**-приложение разворачивается и работает в **Linux**
- → Разобрались в ролях **Docker**, **systemd** и **Nginx**
- → Научились управлять сервисом через **systemd**
- → Научились использовать **journalctl** для анализа логов и ошибок запуска
- → Поняли типовые проблемы при запуске и эксплуатации сервиса

Разворачивание Spring Boot приложений (nginx, systemd, firewall, journalctl)

Виктор Лоскутов
Java Team Lead, Сбер

